LIBRARY MANAGEMENT SYSTEM

Python Midterm Project Report

Team members: Add all three names before submission.
Course: Programming Principles 2
Submission: October 18, 2026

Project description

The Library Management System is a menu-driven Python application for tracking books, members, and borrowing activity. It stores records in SQLite and supports common library tasks through a console menu. The submitted database is preloaded with demonstration records so the program can be run and demonstrated immediately.

Objectives

The program demonstrates functions, conditions, loops, input/output, object-oriented programming, file handling, and relational database operations. It supports Create, Read, Update, and Delete actions, validates input, calculates borrowing statistics from stored records, and exports a CSV summary.

Database structure

Table	Main fields	Relationship
books	book_id, ISBN, title, author, genre, year, available	A book can appear in multiple borrowings
members	member_id, full_name, email, phone	A member can have multiple borrowings
borrowings	borrowing_id, book_id, member_id, dates	Links one book to one member per loan

Foreign keys connect borrowings to books and members. Cascading deletes keep the database consistent when a parent record is removed.

Program design and features

Classes and functions

LibraryManager owns the SQLite connection and groups database operations for adding records, searching, updating, deleting, returning items, calculating statistics, and exporting results. Its connection and database path are private attributes. **Book** and **Member** dataclasses represent the main record types.

The menu uses a loop and small helper functions for integer input and table formatting. It runs until Exit is selected. Validation and database errors are caught and shown as readable messages.

Required operations

Operation	Implementation
Add	Create books and members; create borrowings after checking IDs and availability.
Display	List books, members, and borrowing history in readable columns.
Search	Search books by title and members by name or exact ID. Empty results display “No records found.”
Update	Change a book title, author, genre, and published year.
Delete / cancel	Delete a book or cancel a borrowing; active cancellations restore availability.
Return	Mark an active borrowing returned and make the book available again.
Export	Write summary metrics and borrowing details to results.csv.

Additional programming features

Regular expressions: ISBNs must match a 10- or 13-digit pattern (the 10-digit form can end in X); member emails are validated before saving. **Random:** the demo dataset uses a seeded random number generator for sample return dates, which keeps the sample repeatable.

Input safeguards

The app rejects blank book details, malformed ISBNs or emails, invalid publication years, missing IDs, and attempts to borrow unavailable books. SQLite constraints enforce unique ISBNs and emails, valid availability values, and foreign-key relationships.

Dataset and results

Demonstration dataset

The included project.db contains 50 books, 30 members, and 30 borrowing records, meeting the required minimum for the library topic. Names and contact details use fictional example data. The dataset is repeatable so the defense can use stable figures.

Calculations from the database

Measure	Result	Calculation
Total borrowings	30	Count rows in borrowings
Currently borrowed	20	Count records with no returned_on date
Most frequently borrowed	All titles tied at 1	Group by book and count; alphabetical order chooses the displayed title when tied

Program output captured from the app

```
Choose an option: 9
Book Count: 50
Member Count: 30
Borrowing Count: 30
Active Borrowings: 20
Most Borrowed Title: A Better Algorithm 2
Most Borrowed Count: 1
```

Team contributions

Fill in the actual names and work completed by each student before submission.

Team member	Contribution
Member 1 - add name	Describe implementation work and presentation topics.
Member 2 - add name	Describe database, report, or feature work.
Member 3 - add name	Describe interface, data, or presentation work.

Run from the project folder with python3 main.py. Export a fresh results.csv after changes so submitted results match the current database.